safe_ads

A safe, reusable AdMob layer for Flutter apps
Complete guide for the developer (Nikhil D Suthar)

Version 0.1.0 · published on pub.dev

इस गाइड में आसान भाषा में बताया गया है कि पैकेज क्या है, कैसे बना,
कैसे काम करता है, और किसी भी ऐप में कैसे इस्तेमाल करना है।

pub.dev/packages/safe_ads | github.com/Nikilsuthar/safe_ads

1. What is this package, in one minute

Before: every app needed its own ad code: ask AdMob for an ad, wait, handle failures, retry, ask for consent, don't show too many ads... Writing this again and again in every project is slow, and small mistakes can hurt your AdMob account (too many requests, ads too often, accidental extra requests when the user moves between screens).

Now: all of that lives in **one package**, `safe_ads`. In any app you write about 10 lines: give your ad unit ids, put a banner widget on a screen, and call one function where you want a full-screen ad. The package handles everything else, with **safe defaults**.

> **सरल भाषा में:** पैकेज = एक तैयार टूलबॉक्स। हर ऐप में विज्ञापन का कोड दोबारा लिखने की जरूरत नहीं। आप सिर्फ ID देते हैं और फंक्शन बुलाते हैं; बार-बार रिक्वेस्ट, फेल होने पर रिट्राई, कंसेंट, गैप — सब पैकेज खुद संभालता है।

Where your own app code changed (QRiva)

Before	After
~300 lines of ad code inside QRiva (`AdService` , `InterstitialAdManager` , `BannerAdWidget`)	Deleted. QRiva only calls `SafeAds.init` , `SafeBannerAd` , `SafeAds.registerAction` , `SafeAds.showInterstitial`
Fix a bug = fix it in every app	Fix once in the package, update the version in all apps

2. What a "request" is, and the words you will meet

Word	Meaning (simple)
Ad unit id	A code like `ca-app-pub-123.../456...` from your AdMob dashboard. One id per ad placement/format. It tells AdMob which of your ad slots is asking.
AdMob app id	The id of your whole app (`ca-app-pub-123~456`). It goes into `AndroidManifest.xml` / `Info.plist`, not into Dart code.
Ad request	Your app *asks* AdMob's server: "give me an ad for this ad unit". It costs nothing but it is counted. Too many requests with few ads actually shown = low **match rate**.
Load / Loaded	AdMob answered with an ad and the phone received it. The ad is now *ready* but not yet visible.
Fill / No fill	"No fill" = AdMob had no ad to give for that request (normal for new apps, error code 3). It is not an error in your code.
Impression	The ad was actually **shown** on screen. This is what earns money (together with clicks).
Match rate	Ads delivered ÷ requests made. Wasteful requests lower it.
Invalid traffic	Clicks/impressions Google believes are fake (the owner or friends tapping ads, bots). This can disable your AdMob account. **Never tap your own ads**; use test ads.
Test ads	Google's special ad units that always return dummy ads (label "Test Ad"). Safe to tap. Use them while developing.
Consent (UMP)	In Europe/UK the law requires asking permission before personalised ads. Google's form does this. It must be shown *before* ads are requested.
Cooldown	A minimum waiting time between two full-screen ads.
Back-off	Waiting a little longer after each failure before trying again (30 s, then 60 s).

3. The six ad types, and what each needs

Type	What the user sees	Ad unit id needed	How you use it in code
Banner	A strip at the bottom/top of a screen. Always visible while on the screen.	`banner`	`SafeBannerAd(placement: 'home')`
Native	An ad styled like content (image, title, button) inside a list/screen.	`native`	`SafeNativeAd(placement: 'feed')`
Interstitial	Full-screen ad at a natural pause (after saving a QR, finishing a level).	`interstitial`	`SafeAds.showInterstitial(onFinished: ...)`
Rewarded	Full-screen video the user *chooses* to watch to get a reward (coins, a hint).	`rewarded`	`SafeAds.showRewarded(onReward: ...)`
Rewarded interstitial	Like interstitial, but gives a reward; user is told first.	`rewardedInterstitial`	`SafeAds.showRewardedInterstitial(...)`
App open	Full-screen ad when the user returns to the app from the background.	`appOpen`	automatic with `appOpenEnabled: true`

What every app needs (checklist)

1. An AdMob account, an app in it, and one **ad unit per format** you use.
2. The **AdMob app id** in `AndroidManifest.xml` (and `Info.plist` for iOS). A wrong format here crashes the app at start.
3. `INTERNET` permission (Android).
4. The package in `pubspec.yaml`: `safe_ads: ^0.1.0`.
5. In Play Console: declare "contains ads", the Advertising ID, and the AdMob data in Data safety. Publish a privacy policy.
6. Use **test ids** while testing (`useTestIds: true`).

4. How a developer uses it (the complete example)

```
// pubspec.yaml
dependencies:
  safe_ads: ^0.1.0

// main.dart
await SafeAds.init(const SafeAdsConfig(
  ids: AdIds(
    banner:       'ca-app-pub-XXXX/1111111111',
    interstitial: 'ca-app-pub-XXXX/2222222222',
    rewarded:     'ca-app-pub-XXXX/3333333333',
  ),
  useTestIds: kDebugMode,                // test ads in debug builds
  autoPreload: {AdFormat.interstitial},  // load one in advance
));
runApp(const MyApp());

// any screen: a banner
bottomNavigationBar: const SafeBannerAd(placement: 'home'),

// after the user finished something
await SafeAds.registerAction();
await SafeAds.showInterstitial(
  placement: 'saved',
  onFinished: () => Navigator.push(...),   // runs ALWAYS, exactly once
);

// rewarded: user taps "Watch ad for a hint"
await SafeAds.showRewarded(
  onReward: (reward) => giveHint(),
  onNotAvailable: () => toast('Try again in a moment'),
);
```

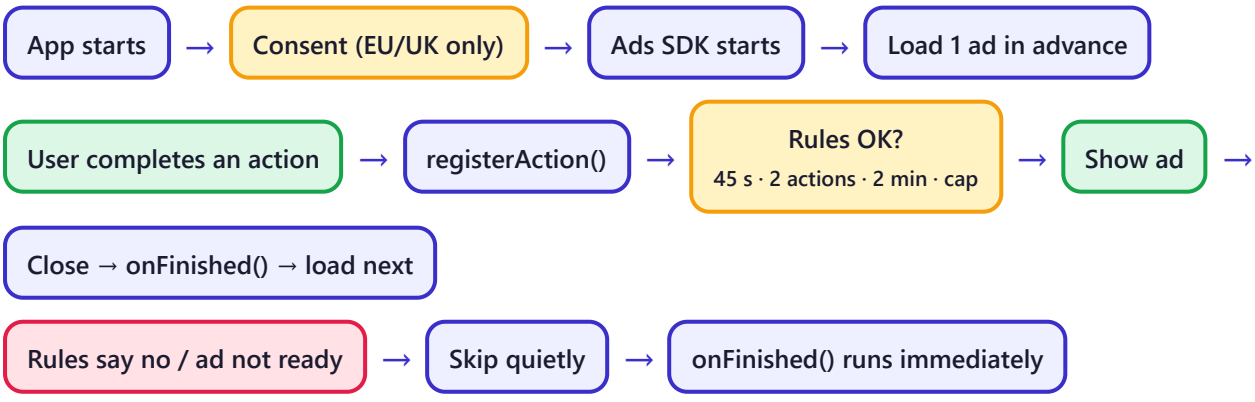
> **Important:** `onFinished` runs after the ad closes *or immediately if no ad was shown*. So your navigation always continues — the user is never stuck because an ad failed.

5. The safety rules (what the package does for you)

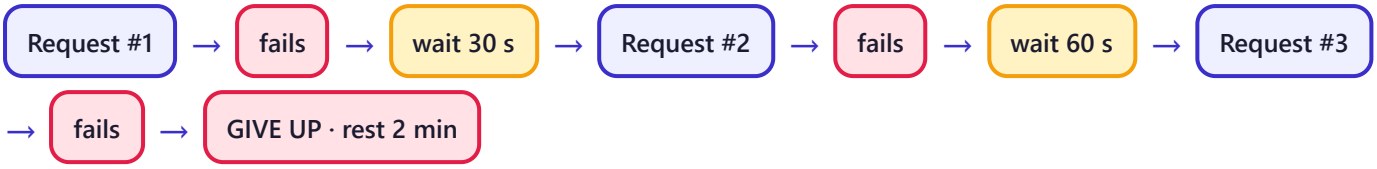
All numbers can be changed with `AdPolicy`; the defaults are deliberately cautious.

Rule	Default	Why it exists
Retries after a failed load	1 try + 2 retries (after **30 s**, then **60 s**) = **3 requests** max	A failure is usually "no ad now". Hammering AdMob doesn't help and lowers match rate.
Rest after giving up	**2 minutes** (10 min if the ad unit id itself is invalid)	Stops endless loops. A new screen visit or user action later starts a fresh cycle.
Hourly request cap	**120 requests per hour** for the whole app	A safety net: even if a bug or fast navigation causes loops, requests can never explode.
Banner / native reuse	Re-shown for **5 minutes** when you leave and come back	No new request on every visit. After 5 minutes it is considered old and a fresh one is requested.
Interstitial pacing	After **2 completed actions**, **2 min** cooldown, not in the first **45 s**, max **10 per session**, never two at once	Google's ad-placement policy: ads at natural breaks, never spammy. Protects users and your account.
"1 hour discard" for full-screen ads	A loaded interstitial/rewarded ad older than **55 minutes** is thrown away and a new one loaded	AdMob stops accepting full-screen ads about **1 hour** after loading. Showing an expired ad fails. So the package discards it before that, and loads a new one.
App-open ad	Off by default. If on: after 30 s in background, 5 min between ads, not right after another ad	Avoids ads after a permission dialog or share sheet, which would annoy users.
Counters survive restart	Saved with `shared_preferences`	Closing and reopening the app can't be used to bypass the cooldown.

Flow of one full-screen ad (interstitial)



Flow when loading fails



During the rest, nothing is requested. This was proven on a real phone: with wrong ad ids, each slot made exactly 3 requests and then stopped.

Banner on screens



Rebuilding a screen (setState, theme change) never requests an ad. Only a **new visit** can, and only if there is no fresh ad to reuse.

6. How the package was built

Structure

File	Job
`lib/safe_ads.dart`	The public door. Exports only what developers should use.
`src/config.dart`	Settings classes: `SafeAdsConfig`, `AdIds` (incl. Google's test ids), `AdPolicy` (all the numbers), events and results.
`src/guards.dart`	Pure safety logic: `RequestBudget` (120/hour), `RetryTracker` (3 tries then rest), `InterstitialGate` (cooldown/actions). No ads here, so it is easy to unit-test.
`src/core.dart`	Shared state: logging, statistics, the ready flag, `AdPool` (keeps banners for reuse).
`src/consent.dart`	Google's consent (UMP) flow with time limits so a slow network can never freeze the app.
`src/full_screen.dart`	One "slot" that preloads a full-screen ad, retries, expires old ads and hands it over to show.
`src/safe_ads.dart`	The `SafeAds` class: `init`, `showInterstitial`, `showRewarded`, ... and the app-open observer.
`src/widgets.dart`	`SafeBannerAd` and `SafeNativeAd` widgets (reuse + retry + budget).
`src/prefs_storage.dart`	Saves the counters between app runs.
`example/`	A test-bench app with a button for every ad type and a live log.
`test/`	16 unit tests for the safety rules.

What it uses from other packages

Package	Used for
google_mobile_ads (by Google)	The real work of talking to AdMob: `BannerAd`, `InterstitialAd`, `RewardedAd`, `NativeAd`, `AppOpenAd`, and the consent classes. `safe_ads` *wraps* it — it adds rules and safety on top, it does not replace it. (Versions 5.3.1 to 6.x.)
shared_preferences	Remember the action counter and the time of the last interstitial.
flutter (SDK)	Widgets, `ValueNotifier`, app life-cycle (for app-open ads).

> Nothing is copied from another package's code. `safe_ads` is an **unofficial wrapper**; Google's package does the ad delivery. The README says it is not affiliated with Google.

Design decisions (why it is "safe")

- **Never throws, never blocks.** Ad failures are logged and ignored; the app keeps working.
- **One entry point** (`SafeAds`) so rules cannot be bypassed by calling the ad SDK from random places.
- **Everything is counted** (stats + events) so you can *see* what happened.
- **Guards are separate from ads** so the rules can be unit-tested without a phone.
- **Clock injection** in the guard classes lets tests move time forward without waiting.

7. How to check that it works (logs)

In debug builds every event is printed. Filter with:

```
adb logcat -s flutter | findstr SafeAds
```

Log line	Meaning
`request banner @home`	The app asked AdMob for a banner for the screen "home".
`loaded banner @home`	An ad arrived and is ready/visible.
`impression banner @home`	The ad was actually shown.
`reused banner @home - showing the ad from the previous visit`	No new request; the previous ad was shown again.
`failed ... code 3: No fill`	AdMob had no ad right now (normal).
`retryScheduled ... retry 1/2 in 30s`	First automatic retry planned.
`gaveUp ... resting 120s`	3 attempts used; the slot rests, no more requests for 2 minutes.
`skipped interstitial - notEnoughActions / cooldown / launchGrace`	A rule blocked the interstitial (this is correct behaviour).
`blocked ... hourly request budget`	The 120-per-hour cap was reached.
`reward rewarded - 10 coins`	The user earned the reward.

In code: `SafeAds.stats.summary()` prints requests / loaded / failed / shown / reused per format; `SafeAds.events` is a live stream; `SafeAds.recentEvents` gives the last 300 events for a debug screen.

What was tested on a real phone (Moto G51, Google test ads)

Test	Result
All 6 formats load and show	Passed
Interstitial with 0 actions	Skipped (`notEnoughActions`), `onFinished` still ran
Interstitial after 2 actions	Shown, next one preloaded after closing
Interstitial again within the cooldown, even after restarting the app	Blocked (`cooldown`)
Rewarded and rewarded interstitial	Shown, reward "10 coins" granted
App-open ad	Shown
3 visits to a banner screen	1 request + 2 reuses
Wrong ad ids	Exactly 3 requests per slot, then rest, then nothing
QRiva using the package	Banner + interstitial worked; 3 visits = 1 request

> **Not tested:** iOS on a real iPhone (no Mac available). The Dart code is shared and iOS uses Google's documented test ids, but it has not been run there. A native-ad test creative shows a cosmetic Google debug warning.

8. Staying safe with your AdMob account

- **Never tap your own real ads** — not even once "to check". Use `useTestIds: true` or add your phone to `testDeviceIds` .
- Do not ask friends/testers to tap ads. Their normal accidental taps are fine; organised tapping is not.
- Don't put banners next to buttons people tap often; don't show interstitials at app launch, on exit or in the middle of a task.
- New apps show few ads at first (AdMob "Getting ready" review takes days) — that is normal and not a bug.
- Keep ads declared in Play Console (ads, Advertising ID, Data safety) and keep your privacy policy online.

9. Using it in another project (step by step)

1. Add `safe_ads: ^0.1.0` to `pubspec.yaml` and run `flutter pub get` .
2. In AdMob create the ad units you need; copy the **app id** and **ad unit ids**.
3. Put the app id in `AndroidManifest.xml` (`com.google.android.gms.ads.APPLICATION_ID`) and, for iOS, `GADApplicationIdentifier` in `Info.plist` .

4. Call `SafeAds.init(...)` once in `main()` (use test ids while developing).
5. Add `SafeBannerAd` where a banner should appear; call `registerAction` + `showInterstitial` at natural breaks.
6. Run the app, watch the logs, then switch to real ids for your release build.
7. Before release: Play Console declarations, privacy policy, and a Europe consent message in AdMob (Privacy & messaging).

10. About pub.dev

- The package is live at **pub.dev/packages/safe_ads** (version 0.1.0). Its score is 150 of 160 points.
- **Why it may not appear in a list/search page yet:** pub.dev search ranks by relevance, *likes*, *popularity (downloads)* and score. A brand-new package has 0 likes and 0 downloads, so it sits far down general searches like "admob". Searching its exact name `safe_ads` does find it. Visibility grows with likes and downloads, and the search index also updates with a delay of hours to a day.
- A published version can never be deleted; to change anything, publish a new version (0.1.1, 0.2.0...).

Releasing an update

1. Change the code, raise `version:` in pubspec.yaml, add a line to CHANGELOG.md.
2. `flutter analyze` → `flutter test` → `flutter pub publish --dry-run` → `flutter pub publish`.
3. In each app: change `safe_ads: ^0.1.0` to the new version and run `flutter pub get`.

11. Quick glossary in Hindi

शब्द	मतलब
Ad request	ऐप का AdMob से विज्ञापन माँगना। ज़्यादा माँगना और कम दिखाना = खराब match rate।
Impression	विज्ञापन सच में स्क्रीन पर दिखा। कमाई यहीं से होती है।
No fill	AdMob के पास अभी दिखाने को विज्ञापन नहीं था — सामान्य बात है।
Cooldown	दो full-screen विज्ञापनों के बीच कम से कम इतना इंतज़ार।
Retry / Back-off	फेल होने पर थोड़ी देर रुककर दोबारा कोशिश (30 सेकंड, फिर 60 सेकंड), फिर बंद।
1 घंटे वाला discard	लोड हुआ full-screen विज्ञापन करीब 1 घंटे बाद expire हो जाता है, इसलिए 55 मिनट बाद पैकेज उसे हटाकर नया लोड करता है।
Hourly cap	एक घंटे में अधिकतम 120 रिक्वेस्ट — लूप या बग से अकाउंट को बचाने की सुरक्षा।
Test ads	Google के नकली विज्ञापन, जिन पर टैप करना सुरक्षित है।
Invalid traffic	अपने या दोस्तों के नकली क्लिक — इससे AdMob अकाउंट बंद हो सकता है।

Generated for Nikhil D Suthar · safe_ads 0.1.0 · MIT license · unofficial wrapper around google_mobile_ads, not affiliated with Google.